

Algoritmos de optimización - Seminario

Nombre y Apellidos:

Url: <https://github.com/.../03MAIR---Algoritmos-de-Optimizacion---2019/tree/master/SEMINARIO>

Problema:

1. Sesiones de doblaje
2. Organizar los horarios de partidos de La Liga
3. Combinar cifras y operaciones

Descripción del problema:(copiar enunciado)

1. Sesiones de doblaje

(*) La respuesta es obligatoria

Sesiones de doblaje

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

Las posibilidades sin tener en cuenta las restricciones sería:

$$5^{30} = 931322574615478515625$$

Las posibilidades tomando en cuenta las restricciones sería:

$$\frac{30!}{6^5 * 5!} = 58905214586600$$

Modelo para el espacio de soluciones

(*) ¿Cuál es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

La estructura es una matriz binaria, esta permite el acceso a la información de forma sencilla $O(1)$, además como los valores son representados con 1 y 0 es de bajo costo computacional (1 si participa o 0 si no participa en la escena).

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

La funcion objetivo seria

$$\min \sum_d \sum_a y_{a,d}$$

```
In [1]: import itertools
import math
import numpy as np
import random
import copy
```

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

```
In [ ]: # 1. DEFINICIÓN DE LOS DATOS DE ENTRADA
# Matriz de 30 tomas (filas) y 10 actores (columnas)
matriz = [
    [1,1,1,1,1,0,0,0,0,0],
    [0,0,1,1,1,0,0,0,0,0],
    [0,1,0,0,1,0,1,0,0,0],
    [1,1,0,0,0,0,1,1,0,0],
    [0,1,0,1,0,0,0,1,0,0],
    [1,1,0,1,1,0,0,0,0,0],
    [1,1,0,1,1,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,0,0],
    [1,1,0,1,0,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,1,0],
    [1,1,1,0,1,0,0,1,0,0],
    [1,1,1,1,0,1,0,0,0,0],
    [1,0,0,1,1,0,0,0,0,0],
    [1,0,1,0,0,1,0,0,0,0],
    [1,1,0,0,0,0,1,0,0,0],
    [0,0,0,1,0,0,0,0,0,1],
    [1,0,1,0,0,0,0,0,0,0],
    [0,0,1,0,0,1,0,0,0,0],
    [1,0,1,0,0,0,0,0,0,0],
    [1,0,1,1,1,0,0,0,0,0],
    [0,0,0,0,0,1,0,1,0,0],
    [1,1,1,1,0,0,0,0,0,0],
    [1,0,1,0,0,0,0,0,0,0],
    [0,0,1,0,0,1,0,0,0,0],
    [1,1,0,1,0,0,0,0,0,1],
    [1,0,1,0,1,0,0,0,1,0],
    [0,0,0,1,1,0,0,0,0,0],
```

```

[1,0,0,1,0,0,0,0,0,0],
[1,0,0,0,1,1,0,0,0,0],
[1,0,0,1,0,0,0,0,0,0]
]

MAX_TOMAS_POR_DIA = 6
NUM_ACTORES = 10

# Variables globales para registrar la mejor solución
menor_costo_global = math.inf
mejor_planificacion = []

def calcular_costo_dia(dia_tomas):
    """Calcula cuántos actores únicos se necesitan para un grupo de tomas."""
    actores_presentes = set()
    for toma in dia_tomas:
        for actor_id in range(NUM_ACTORES):
            if matriz[toma][actor_id] == 1:
                actores_presentes.add(actor_id)
    return len(actores_presentes)

def fuerza_bruta_doblaje(tomas_disponibles, planificacion_actual, costo_actual):
    """Algoritmo recursivo que busca la combinación óptima."""
    global menor_costo_global, mejor_planificacion

    # PODA: Si el costo que llevamos ya supera al mejor, no seguimos por aquí, si q
    if costo_actual >= menor_costo_global:
        return

    # CASO BASE: Si ya no hay más tomas que asignar
    if not tomas_disponibles:
        if costo_actual < menor_costo_global:
            menor_costo_global = costo_actual
            mejor_planificacion = list(planificacion_actual)
        return

    # Convertimos a Lista para fijar un elemento inicial y evitar duplicidad de com
    lista_disponibles = list(tomas_disponibles)
    toma_base = lista_disponibles[0]
    resto_tomas = lista_disponibles[1:]

    # Generamos subconjuntos de tamaño 0 hasta 5 para acompañar a la toma_base (Máx
    for r in range(min(MAX_TOMAS_POR_DIA, len(tomas_disponibles))):
        for combo in itertools.combinations(resto_tomas, r):
            dia_nuevo = (toma_base,) + combo
            costo_dia = calcular_costo_dia(dia_nuevo)

            # Preparar elementos para la siguiente llamada recursiva
            planificacion_actual.append(dia_nuevo)
            proximas_disponibles = tomas_disponibles - set(dia_nuevo)

            # Llamada recursiva
            fuerza_bruta_doblaje(proximas_disponibles, planificacion_actual, costo_

```

```

        # Backtracking
        planificacion_actual.pop()

# EJECUCIÓN DEL ALGORITMO
if __name__ == "__main__":
    # Creamos un conjunto con los índices de las tomas (0 a 29)
    todas_las_tomas = set(range(len(matriz)))

    print("Calculando la planificación óptima por fuerza bruta...")
    fuerza_bruta_doblaje(todas_las_tomas, [], 0)

    # 3. MOSTRAR RESULTADOS
    print("\n=== RESULTADO ÓPTIMO ENCONTRADO ===")
    print(f"Menor costo total de actores: {menor_costo_global}")
    print("Distribución de las sesiones por día:")

    for i, dia in enumerate(mejor_planificacion, 1):
        # Sumamos 1 a los índices para mostrarlos en formato humano (Tomas 1 a 30)
        tomas_humanas = [t + 1 for t in dia]
        costo_dia = calcular_costo_dia(dia)
        print(f"  Día {i}: Tomas {tomas_humanas} -> Actores necesarios: {costo_dia}")

```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

La complejidad por fuerza bruta sería de:

$$O(c^n)$$

(*)Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta.

Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

```

In [ ]: matriz = [
    [1,1,1,1,1,0,0,0,0,0],
    [0,0,1,1,1,0,0,0,0,0],
    [0,1,0,0,1,0,1,0,0,0],
    [1,1,0,0,0,0,1,1,0,0],
    [0,1,0,1,0,0,0,1,0,0],
    [1,1,0,1,1,0,0,0,0,0],
    [1,1,0,1,1,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,0,0],
    [1,1,0,1,0,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,1,0],
    [1,1,1,0,1,0,0,1,0,0],
    [1,1,1,1,0,1,0,0,0,0],
    [1,0,0,1,1,0,0,0,0,0],
    [1,0,1,0,0,1,0,0,0,0],
    [1,1,0,0,0,0,1,0,0,0],
    [0,0,0,1,0,0,0,0,0,1],

```

```

[1,0,1,0,0,0,0,0,0,0],
[0,0,1,0,0,1,0,0,0,0],
[1,0,1,0,0,0,0,0,0,0],
[1,0,1,1,1,0,0,0,0,0],
[0,0,0,0,0,1,0,1,0,0],
[1,1,1,1,0,0,0,0,0,0],
[1,0,1,0,0,0,0,0,0,0],
[0,0,1,0,0,1,0,0,0,0],
[1,1,0,1,0,0,0,0,0,1],
[1,0,1,0,1,0,0,0,1,0],
[0,0,0,1,1,0,0,0,0,0],
[1,0,0,1,0,0,0,0,0,0],
[1,0,0,0,1,1,0,0,0,0],
[1,0,0,1,0,0,0,0,0,0]
]

NUM_TOMAS = 30
NUM_DIAS = 5
MAX_TOMAS_DIA = 6

def costo_dia(tomas_dia):
    actores = set()

    for toma in tomas_dia:
        fila = matriz[toma - 1]

        for actor, requerido in enumerate(fila):
            if requerido == 1:
                actores.add(actor)

    return len(actores)

def costo_total(solucion):
    return sum(costo_dia(dia) for dia in solucion)

def generar_solucion_aleatoria():
    tomas = list(range(1, NUM_TOMAS + 1))
    random.shuffle(tomas)

    solucion = []

    for i in range(NUM_DIAS):
        inicio = i * MAX_TOMAS_DIA
        fin = inicio + MAX_TOMAS_DIA
        solucion.append(tomas[inicio:fin])

    return solucion

def generar_vecinos(solucion):
    vecinos = []

    for d1 in range(NUM_DIAS):
        for d2 in range(d1 + 1, NUM_DIAS):

```

```

        for t1 in solucion[d1]:

            for t2 in solucion[d2]:

                nuevo = copy.deepcopy(solucion)

                idx1 = nuevo[d1].index(t1)
                idx2 = nuevo[d2].index(t2)

                nuevo[d1][idx1] = t2
                nuevo[d2][idx2] = t1

                vecinos.append(nuevo)

    return vecinos

def hill_climbing(solucion_inicial):

    actual = solucion_inicial
    costo_actual = costo_total(actual)

    mejora = True

    while mejora:

        mejora = False

        vecinos = generar_vecinos(actual)

        mejor_vecino = actual
        mejor_costo = costo_actual

        for vecino in vecinos:

            costo = costo_total(vecino)

            if costo < mejor_costo:
                mejor_vecino = vecino
                mejor_costo = costo
                mejora = True

        actual = mejor_vecino
        costo_actual = mejor_costo

    return actual, costo_actual

def random_restart(iteraciones=1000):

    mejor_solucion = None
    mejor_costo = float("inf")

    for i in range(iteraciones):

        inicial = generar_solucion_aleatoria()

```

```

    solucion, costo = hill_climbing(inicial)

    if costo < mejor_costo:
        mejor_costo = costo
        mejor_solucion = solucion

        print(f"Iteración {i+1}: Nuevo mejor costo = {costo}")

    return mejor_solucion, mejor_costo

solucion, costo = random_restart(1000)

print("\n" + "="*50)
print("MEJOR SOLUCIÓN ENCONTRADA")
print("="*50)

for dia, tomas in enumerate(solucion, start=1):

    actores = set()

    for toma in tomas:
        for actor, requerido in enumerate(matriz[toma-1]):
            if requerido:
                actores.add(actor + 1)

    print(f"\nDía {dia}")
    print("Tomas:", sorted(tomas))
    print("Actores:", sorted(actores))
    print("Cantidad actores:", len(actores))

print("\nCosto total:", costo)

```

Iteración 1: Nuevo mejor costo = 30
Iteración 9: Nuevo mejor costo = 29
Iteración 17: Nuevo mejor costo = 28
Iteración 475: Nuevo mejor costo = 27

=====
MEJOR SOLUCIÓN ENCONTRADA
=====

Día 1
Tomas: [8, 10, 11, 21, 26, 29]
Actores: [1, 2, 3, 5, 6, 8, 9]
Cantidad actores: 7

Día 2
Tomas: [3, 4, 5, 15, 16, 25]
Actores: [1, 2, 4, 5, 7, 8, 10]
Cantidad actores: 7

Día 3
Tomas: [7, 9, 13, 27, 28, 30]
Actores: [1, 2, 4, 5]
Cantidad actores: 4

Día 4
Tomas: [1, 2, 6, 12, 20, 22]
Actores: [1, 2, 3, 4, 5, 6]
Cantidad actores: 6

Día 5
Tomas: [14, 17, 18, 19, 23, 24]
Actores: [1, 3, 6]
Cantidad actores: 3

Costo total: 27

(*)Calcula la complejidad del algoritmo

Respuesta

La complejidad por Hill Climbing con Random Restart seria de:

$$O(n^3)$$

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

```
In [ ]: matriz = np.zeros((30, 10), dtype=int)
tomas_por_dia = {0:0,1:0,2:0,3:0,4:0,5:0,6:0,7:0,8:0,9:0}

def numero_random():
    # Generacion del número random
```



```

    rng = np.random.default_rng()
    return rng.integers(0, 10)

def generar_dia():
    # Generacion dia completo
    tomas = []
    valor = numero_random()
    contador = 0
    while len(tomas) < 5:
        numero = numero_random()
        if numero not in tomas:
            tomas.append(numero)
            contador += 1
            if contador == valor:
                return tomas
    return tomas

def randomMatriz():
    for i in range(30):
        tomas = generar_dia()
        for toma in tomas:
            matriz[i][toma] = 1
            tomas_por_dia[toma] = tomas_por_dia[toma] + 1 #Informacion de cuantas t
    return matriz
matriz = randomMatriz()
print(matriz)

```

```

[[0 0 0 1 0 1 0 0 1 1]
 [0 0 0 0 1 0 0 0 1 0]
 [0 0 1 0 0 0 0 1 0 0]
 [0 0 0 1 1 1 1 1 0 0]
 [0 0 0 0 0 1 1 0 0 1]
 [0 1 0 1 1 1 0 0 0 1]
 [0 0 1 0 0 0 0 0 0 0]
 [1 0 0 1 1 1 1 0 0 0]
 [1 0 0 0 0 0 1 0 0 0]
 [1 1 1 1 0 0 1 0 0 0]
 [0 1 0 1 0 1 1 1 0 0]
 [1 0 1 0 1 1 0 0 0 0]
 [0 1 0 1 0 0 0 0 1 0]
 [0 1 0 0 0 0 0 0 0 1]
 [1 0 0 0 1 0 0 0 1 1]
 [0 0 0 0 0 1 0 0 1 0]
 [0 0 0 0 1 0 0 0 0 0]
 [1 0 0 1 1 0 1 0 1 0]
 [1 1 0 0 0 0 1 1 1 0]
 [1 0 0 1 0 1 0 1 1 0]
 [1 1 0 1 0 1 0 0 1 0]
 [0 0 0 0 0 0 1 0 0 0]
 [0 0 1 1 1 1 0 0 0 1]
 [0 0 1 0 0 0 0 0 0 0]
 [0 1 0 1 1 1 0 1 0 0]
 [0 1 1 1 0 0 0 1 0 1]
 [0 1 0 0 1 0 1 0 0 1]
 [0 0 1 1 0 0 0 1 1 1]
 [0 0 0 0 1 1 0 0 0 1]
 [0 1 0 1 0 1 1 0 0 1]]

```

Aplica el algoritmo al juego de datos generado

Respuesta

```

In [4]: # =====
# EJECUCIÓN
# =====

solucion, costo = random_restart(1000)

print("\n" + "="*50)
print("MEJOR SOLUCIÓN ENCONTRADA")
print("="*50)

for dia, tomas in enumerate(solucion, start=1):

    actores = set()

    for toma in tomas:
        for actor, requerido in enumerate(matriz[toma-1]):
            if requerido:
                actores.add(actor + 1)

    print(f"\nDía {dia}")

```

```
print("Tomas:", sorted(tomas))
print("Actores:", sorted(actores))
print("Cantidad actores:", len(actores))

print("\nCosto total:", costo)
```

Iteración 1: Nuevo mejor costo = 37
Iteración 5: Nuevo mejor costo = 36
Iteración 70: Nuevo mejor costo = 35
Iteración 430: Nuevo mejor costo = 34

=====
MEJOR SOLUCIÓN ENCONTRADA
=====

Día 1
Tomas: [3, 7, 13, 24, 26, 28]
Actores: [2, 3, 4, 8, 9, 10]
Cantidad actores: 6

Día 2
Tomas: [5, 14, 17, 22, 27, 29]
Actores: [2, 5, 6, 7, 10]
Cantidad actores: 5

Día 3
Tomas: [4, 11, 19, 20, 21, 25]
Actores: [1, 2, 4, 5, 6, 7, 8, 9]
Cantidad actores: 8

Día 4
Tomas: [6, 8, 10, 12, 23, 30]
Actores: [1, 2, 3, 4, 5, 6, 7, 10]
Cantidad actores: 8

Día 5
Tomas: [1, 2, 9, 15, 16, 18]
Actores: [1, 4, 5, 6, 7, 9, 10]
Cantidad actores: 7

Costo total: 34

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Respuesta:

Declaración sobre el uso de inteligencia artificial. En este trabajo se utilizó ChatGPT (OpenAI, modelo GPT-5.5) como herramienta de apoyo para la generación de ideas, la revisión y la mejora. El contenido final, las verificaciones técnicas y las conclusiones fueron elaborados y validados por el autor.

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Respuesta

Con este problema vemos que a medida que aumenta el número de tomas y actores, el problema crece de forma combinatoria, haciendo muy difícil o inviable explorar todas las soluciones posibles mediante búsqueda exhaustiva. Por lo que, una forma de trabajar este problema es desarrollar algoritmos más eficientes, como metaheurísticas (búsqueda local, recocido simulado, búsqueda tabú o algoritmos genéticos) que permitan obtener soluciones de buena calidad en tiempos razonables para instancias de mayor tamaño.

Para este trabajo utilice una búsqueda local Hill Climbing con Random Restart, ya que permite tener soluciones buenas en un tiempo aceptable, sin la necesidad de explorar todo el espacio de soluciones. Con el uso del reinicio aleatorio, intentamos solucionar el problema de la búsqueda local, que puede quedar atrapado en los óptimos locales. Con esto tenemos más probabilidades de obtener una mejor respuesta.

En el problema podríamos incrementar la dificultad incluyendo más restricciones de un proyecto real. Ejemplo: disponibilidad de los actores para días específicos, duración de las sesiones o que algunas tomas tengan que realizarse antes que otras. Todo esto haría que el problema fuera más real y por consiguiente aumente su complejidad.

Finalmente, para instancias de gran tamaño, podría estudiarse el comportamiento del algoritmo al aumentar considerablemente el tamaño del problema, así como el uso de procesamiento paralelo. También sería interesante comparar Hill Climbing con otras metaheurísticas, como Simulated Annealing, Tabu Search o Algoritmos Genéticos, para determinar cuál ofrece el mejor equilibrio entre calidad de la solución y tiempo de cálculo.

Finalmente, también podría investigarse el desarrollo de algoritmos híbridos, utilizando Hill Climbing con Random Restart para obtener una solución inicial que posteriormente sea refinada mediante métodos exactos o por otras metaheurísticas, con el objetivo de mejorar aún más la calidad de la planificación.